

# MASI: Memory-Adaptive Inference Framework for Spiking Neural Networks on Edge Devices

Di Yu  
Zhejiang University  
Hangzhou, Zhejiang, China  
yudi2023@zju.edu.cn

Helin Zheng  
Zhejiang University  
Ningbo, Zhejiang, China  
22551146@zju.edu.cn

Changze Lv  
Fudan University  
Shanghai, China  
czlv22@m.fudan.edu.cn

Xin Du\*  
Zhejiang University  
Ningbo, Zhejiang, China  
xindu@zju.edu.cn

Linshan Jiang\*  
National University of Singapore  
Singapore  
linshan@nus.edu.sg

Xiang Liu  
National University of Singapore  
Singapore  
Liu.xiang@u.nus.edu

Gang Pan  
Zhejiang University  
Hangzhou, China  
gpan@zju.edu.cn

Shuiguang Deng\*  
Zhejiang University  
Hangzhou, China  
dengsg@zju.edu.cn

## Abstract

The rapid development of the Internet of Things (IoT) applications necessitates resource-efficient computing paradigms that can unify heterogeneous sensing modalities. Spiking Neural Networks (SNNs) meet this need with their event-driven and energy-efficient processing nature. However, deploying SNNs on mobile and embedded platforms is hindered by strict and fluctuating memory budgets. While prior work explores lightweight model design and system-level memory management, these methods either sacrifice accuracy or incur high runtime overhead due to timestep-dependent dynamics. To tackle these challenges, we propose a memory-adaptive framework MASI that enables efficient on-device SNN inference by combining (1) a fine-grained memory-adaptive layer slicing strategy, (2) a timestep-agnostic scheduler that maximizes memory utilization with minimal fragmentation, and (3) a timestep-aware early-exit mechanism that reduces redundant calculations. Evaluated on diverse workloads and edge devices, MASI can dynamically adapt to runtime memory availability, approximately reducing memory usage by 20.67% and inference latency by 58.53% on average with negligible accuracy loss compared to other feasible on-device implementations under memory constraints.

## CCS Concepts

• Human-centered computing → Ubiquitous and mobile computing; • Computing methodologies → Artificial intelligence.

\*Xin Du, Linshan Jiang and Shuiguang Deng are corresponding authors.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.  
WWW '26, Dubai, United Arab Emirates  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792136>

## Keywords

Spiking Neural Networks, Edge Device, Memory Management, Neuromorphic Computing, Model Inference

## ACM Reference Format:

Di Yu, Helin Zheng, Changze Lv, Xin Du, Linshan Jiang, Xiang Liu, Gang Pan, and Shuiguang Deng. 2026. MASI: Memory-Adaptive Inference Framework for Spiking Neural Networks on Edge Devices. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792136>

## 1 Introduction

The rapid expansion of the Internet of Things (IoT) has fueled the adoption of edge computing [15], enabling intelligent applications such as visual perception [81], acoustic sensing [85], and wireless environment monitoring [7] to run directly on resource-constrained devices. However, the always-on nature of these services necessitates continuous data acquisition, causing conventional sensors to produce redundant streams that both drain power and overwhelm limited compute pipelines [54].

To address this inefficiency, emerging neuromorphic sensors, such as dynamic vision sensors (DVS) [40], artificial cochleae [38], and neuro-radars [84], offer sparse, event-driven outputs with high temporal fidelity. Yet, existing edge intelligence largely depends on artificial neural networks (ANNs) [62], whose dense computation patterns are inherently misaligned with such sparse sensor events, often leading to energy inefficiency and costly conversions into dense frames for compatibility [79].

This gap motivates spiking neural networks (SNNs) [24] as a unified substrate for edge intelligence. Unlike ANNs, SNNs directly process sparse, asynchronous spikes while preserving temporal precision [75, 80], thus avoiding densification overheads and aligning with neuromorphic sensor outputs. Recent studies demonstrate that SNNs can match ANN-level performance across vision, acoustic, and motion sensing tasks [20, 45, 79]. With suitable encoding and training techniques [3, 30], they generalize beyond event-driven inputs, indicating the potential of SNNs as a unified paradigm across

heterogeneous sensing modalities. Together, these properties establish SNNs as a promising foundation for heterogeneous edge intelligence, broadening sensing coverage and potentially improving energy efficiency by discarding redundant zero-bit operations and operating only on one-bit spikes at the edge [71].

However, memory capacity poses a fundamental bottleneck for deploying SNNs on resource-constrained IoT devices such as smartphones and single-board computers. While cloud servers can utilize multiple NVIDIA H100 GPUs (each with 80 GB of memory) [61], even high-end smartphones like the Samsung Galaxy S25 offer only 12 GB [58]. Moreover, the memory available to SNN-based services is limited by the operating system and co-running applications. Consequently, minimizing the memory footprint of SNNs is essential for enabling practical on-device inference in edge scenarios.

To mitigate this limitation, conventional methods reduce the SNN memory footprint via algorithmic-level lightweight techniques, including model compression [44, 72], manually crafted efficient designs [11, 42], and neural architecture search (NAS) [46, 52]. While effective in reducing memory usage, these approaches require extensive tuning efforts and often suffer from accuracy degradation due to reduced model capacity [60, 79].

In practice, however, SNN inference executes on heterogeneous edge devices with dynamically varying memory, as available capacity is shared with other applications and system processes. This variability complicates the applications of model-centric lightweight techniques and limits the effectiveness of conventional memory management schemes. Prior ANN-based work reduces memory pressure with negligible accuracy loss at the systematic level, e.g., by swapping non-urgent data to storage. Layer streaming [55, 83] executes only active layers, while others optimize memory layout [48, 68] or partition layers [6, 37]. Nevertheless, directly applying these ANN-based techniques to on-device SNNs is challenging: (1) a few bottleneck layers dominate peak usage, limiting benefits of swapping smaller layers [41, 79]; (2) timestep-aware execution triggers repeated memory I/O, amplifying swapping and partitioning costs [47]; (3) memory footprints fluctuate across timesteps, undermining static allocation and complicating per-timestep planning with shared memory. These challenges highlight dynamic strategies that adapt to available memory while enabling efficient, accurate on-device SNN inference.

To tackle these challenges, we propose MASI<sup>1</sup>, an adaptive memory management framework for on-device SNNs under dynamic memory constraints. MASI combines (i) a memory-adaptive slicing strategy to partition bottleneck layers, (ii) a timestep-agnostic scheduler that maximizes memory utilization while avoiding the overhead of per-timestep planning, and (iii) a timestep-aware early-exit mechanism that skips redundant timesteps to accelerate inference. Unlike prior methods that rely on extensive model redesign or static memory allocation, MASI dynamically adapts to varying memory budgets with minimal overhead, enabling efficient, responsive, and scalable on-device SNN inference.

We evaluate MASI across smartphones and single-board edge devices using representative SNN models. Results show that MASI achieves significant memory reduction and latency improvement while incurring negligible accuracy loss, consistently delivering

near-optimal trade-offs under diverse memory conditions. For instance, MASI achieves a maximum memory reduction of 70.03% and a latency improvement of 25.69%, while incurring a mere 0.49% accuracy loss compared to the feasible original implementation for Spiking HD (a neuromorphic acoustic sensing task) on Jetson Orin NX, demonstrating a favorable accuracy–efficiency trade-off. The main contributions of this work are summarized as follows:

- We design an adaptive memory management framework, namely MASI, that enables efficient SNN inference under stringent memory constraints on edge devices.
- To address the substantial and imbalanced memory footprint of SNN inference, we propose a fine-grained memory-adaptive slicing strategy, coupled with a timestep-agnostic memory scheduler, to maximize memory utilization and eliminate fragmentation.
- To eliminate redundant computation and accelerate inference with minimal accuracy loss, we further introduce a timestep-aware early-exit mechanism that adaptively terminates inference for easy inputs according to cumulative confidence scores.
- We implement a MASI prototype and evaluate it on representative edge devices across diverse SNN workloads and sensing modalities. Experimental results show that MASI effectively adapts to dynamic memory budgets, achieving an average 20.67% lower memory footprint and 58.53% lower inference latency, with minimal overhead and negligible accuracy loss.

## 2 Background

**Edge Intelligence with SNNs.** SNNs [24, 65], inspired by biological neural processes, promise higher potential energy efficiency compared to conventional ANNs [62]. This advantage stems from their temporal computing paradigm, where neuronal models determine how information is encoded as single-bit spikes across discrete timesteps. Among various neuronal models [27, 32, 49], the Leaky Integrate-and-Fire (LIF) model [23] is the most widely adopted due to its balance between biological plausibility and computational simplicity. Building on this energy-efficiency potential, SNNs are attractive for resource-constrained IoT environments where power budgets are strict [79]. Recent studies have shown that SNNs achieve competitive accuracy with ANNs on classic IoT sensing tasks, such as image recognition [70] and speech processing [69], demonstrating their practicality. More importantly, SNNs naturally excel at processing asynchronous, spike-driven outputs from emerging neuromorphic sensors such as event-based cameras [40] and neuro-radars [84], often surpassing ANNs in both accuracy and efficiency [14]. These properties position SNNs as a promising paradigm for enabling broader and more energy-efficient edge intelligence. **Appendix A.1** provides a controlled comparison between ANNs and SNNs to validate this potential further.

**Memory Constraints for On-Device SNNs.** Memory is the primary bottleneck for on-device SNN deployment, as it determines the feasibility of executing the model. The memory constraint manifests in the following aspects: (1) As network depth increases, the computation and memory demands of SNNs grow linearly. Although modern smartphones provide sufficient computational power for desktop-scale AI models, memory remains a limiting factor [31]. For example, Samsung’s Galaxy S series saw GPU performance rise from 3–4 GFLOPS in the 2010 S1 to hundreds of GFLOPS in

<sup>1</sup><https://github.com/Cajol1e/MASI>

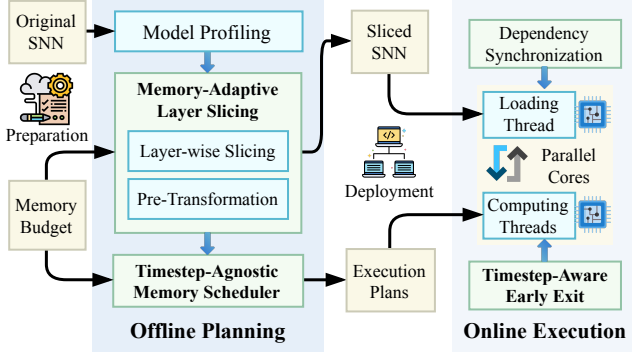


Figure 1: Overall workflow of MASI.

the 2025 S25, while memory capacity increased only from 0.5 GB to 12 GB, revealing an apparent mismatch between compute and memory growth [58]. (2) Edge devices are multi-purpose, and operating systems impose per-app memory limits. For instance, Android restricts heap size (e.g., 512 MB) and uses a low-memory killer [16]. (3) Commercial apps often further constrain memory for specific functions to ensure a smooth user experience, such as Microsoft Editor’s grammar-checker model, which requires less than 50 MB [64]. These challenges motivate the design of an on-device SNN inference framework that prioritizes memory optimization. **Appendix A.2** presents an in-depth memory profiling of a typical SNN model to characterize the memory bottleneck.

### 3 Design of MASI

#### 3.1 Framework Overview

As depicted in Figure 1, we propose MASI, a Memory-Adaptive framework for on-device SNN Inference, which operates in two stages. The first stage is offline planning, where model slicing, loading, and computation are jointly optimized based on the memory budget and the given SNN architecture. The second stage is an online execution phase, where model inference is performed on devices according to the offline plan, augmented with a timestep-aware early-exit mechanism to reduce latency.

In the offline planning stage, MASI first profiles the given SNN model to estimate the per-layer memory requirements and lifetimes of weights, activations, and intermediate results. Based on this analysis, a layer-wise slicing strategy partitions each layer’s computation into appropriately sized segments that comply with the available memory budget. To further reduce runtime overhead during inference, MASI applies weight pre-transformation after slicing, including reshaping and reformatting. This enables the system to directly load transformed weights at runtime, thereby eliminating the need for additional waiting and processing overhead. After layer slicing, a memory scheduler produces a detailed memory layout plan for the sliced SNN model. It employs a lightweight greedy algorithm combined with a preloading strategy to minimize memory fragmentation and I/O waiting time at runtime, while avoiding substantial adaptation overhead.

The online execution stage efficiently conducts SNN model inference, following the offline-derived plans and pre-transformed weights. To ensure stable execution, each layer is decomposed into weight-loading and computation tasks, scheduled according to

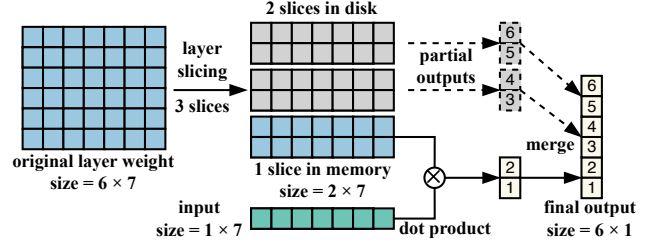


Figure 2: Example of layer slicing on a  $6 \times 7$  fully connected layer, reducing memory usage by 67%.

strict layer dependencies, and executed in parallel across different cores. Moreover, since SNN computation unfolds recurrently over timesteps, MASI incorporates the inference process with a timestep-aware early-exit mechanism that adapts to different input samples and mitigates the latency overhead of sequentially executing sliced layers across timesteps under memory constraints.

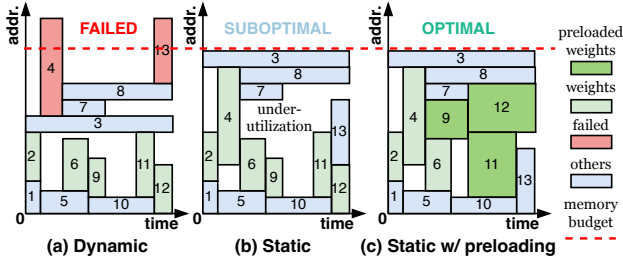
At initialization, MASI performs comprehensive profiling of the given SNN model and determines the optimal execution plan under the current memory budget. When the memory budget changes, MASI first checks whether the existing slicing scheme still satisfies the new constraint. If it does, the framework bypasses redundant slicing to avoid overhead and only updates the memory plan; otherwise, re-slicing is performed to adapt to the new budget.

#### 3.2 Memory-Adaptive Layer Slicing Strategy

Deep SNN models often exhibit imbalanced memory footprints across layers, with deployment bottlenecks typically arising from the peak memory consumption of specific layers [79]. Two primary contributors to these bottlenecks are (1) layer weights and (2) flattened intermediates in specific layers (e.g., inputs to the Im2col+GEMM convolution kernel [8]). Layer slicing provides an intuitive solution to mitigate peak memory usage and enable the deployment of deep networks. As illustrated in Figure 2, this strategy partitions the heavy computational load into multiple slices, computes each slice independently, and merges their outputs to obtain the final result, thereby reducing peak memory consumption.

To reduce the memory footprint caused by excessive layer weight parameters, MASI splits the corresponding layer into multiple sub-layers in the computational graph, each representing a weight slice. Since these weights remain constant during inference, they are initially stored on disk. When required for computation, each sub-layer’s weights are sequentially loaded into memory and multiplied with the input to produce partial inference outputs, thereby reducing memory usage. The partial outputs are then merged to obtain the final result. Weight slicing is particularly suitable for weight-dominated layers, such as fully connected and Winograd convolution layers [17]. The number of weight slices should be minimized under the memory budget, as the slicing approach divides an enormous computational task into multiple smaller ones interleaved with processing, which can substantially increase I/O scheduling overhead when the number of slices is large.

In contrast, the slicing strategy for flattened intermediates partitions the intermediates into multiple slices, which are sequentially multiplied with the in-memory weights to produce partial outputs.



**Figure 3: A static memory management approach with a preloading mechanism can simultaneously reduce memory fragmentation and I/O waiting time. Numbers within the blocks indicate the allocation order.**

These partial outputs are then merged to obtain the final result. Under this strategy, the number of slices should be maximized within device-dependent memory constraints, since input slices are generated directly in memory rather than swapped from disk, thereby avoiding additional I/O scheduling overhead. This approach is particularly suitable for intermediate-dominated layers, such as LIF neuronal layers with membrane potential records and convolution layers implemented with Im2col+GEMM kernels.

Before applying layer slicing, MASI considers the latency–memory trade-off. For convolution kernels, a latency-first policy is followed: the memory footprint of latency-optimized kernels (e.g., Winograd) is first estimated under the slicing approach. If the footprint exceeds the budget, MASI switches to memory-efficient alternatives (e.g., Im2col+GEMM or direct kernels). After slicing, a pre-transformation step reshapes and reformats the layer slices, while also preparing the additional input/output transformations required by kernels such as Winograd. This allows MASI to preload transformed weights, thereby avoiding runtime processing overhead.

### 3.3 Timestep-Agnostic Memory Scheduler

Traditional dynamic on-demand memory management strategies in inference frameworks often suffer from increased memory fragmentation due to the lack of global memory information. Conversely, static approaches typically neglect preloading, resulting in suboptimal memory utilization. Although the memory usage patterns of SNNs resemble those of ANNs, they differ in that LIF neurons must maintain and update membrane potentials across all timesteps, imposing additional memory demands. Inspired by FlexNN [41], MASI adopts a timestep-agnostic static memory management strategy with a preload-enabled scheduler, effectively reducing fragmentation and minimizing I/O waiting time during SNN inference. For clarity, we firstly formulate the memory planning problem for SNNs, followed by a detailed description of the memory planning algorithm employed in MASI.

**Memory Planning Formulation.** Memory planning is commonly formulated as a two-dimensional bin packing (2DBP) problem with a fixed order coordinate [48, 68]. In this formulation, each operator is represented as a rectangle (a.k.a., a “block”) on a two-dimensional plane defined by memory address (y-axis) and execution order (x-axis). The x-range of a block corresponds to its lifecycle, i.e., the logical time from allocation to de-allocation, while the y-range denotes the memory space it occupies. The objective of memory layout

#### Algorithm 1: Memory Planning Scheduler for SNN

---

**Input:** Memory budget  $M_{\max}$ ; block list  $\mathcal{B} = \{(s_i, e_i, m_i, t_i)\}$   
**Output:** Memory layout plan  $\Omega = \{(time_i, addr_i)\}$

- 1 Initialize plan buffer  $\Omega \leftarrow \emptyset$ ;
- 2  $\Omega \leftarrow \text{PLANMEMBRANEPOTENTIAL}(\Omega, \{m_i\}, M_{\max})$ ;
- 3  $\Omega \leftarrow \text{PLANACTIVATIONS}(\Omega, \{(s_i, e_i, m_i)\}, M_{\max})$ ;
- 4 **for each block  $i$  in remained block list  $\mathcal{B}$  do**
- 5     Backup plan buffer:  $\Omega' \leftarrow \Omega$ ;
- 6      $\Omega \leftarrow \text{PLANWITHPRELOAD}(\Omega, \{(s_i, e_i, m_i)\}, M_{\max})$ ;
- 7     **if  $\Omega$  fails then**
- 8         Restore plan buffer:  $\Omega \leftarrow \Omega'$ ;
- 9          $\Omega \leftarrow \text{PLANNOPRELOAD}(\Omega, \{(s_i, e_i, m_i)\}, M_{\max})$  ;
- 10         // Re-schedule blocks w/o preloading.
- 11         **if  $\Omega$  fails then**
- 12             **return Error** ;                     // Schedule fails
- 13 **return  $\Omega$**  ;                                     // Schedule succeeds

---

planning is to determine an optimal placement of all blocks with known lifecycles and memory requirements. Our memory planning problem can be formulated as a variant of the 2DBP problem. The key distinction is that, in our formulation, both the allocated memory addresses and the allocation times of blocks are determined by the planning outcome, potentially resulting in longer lifecycles compared to the original formulation, as illustrated in Figure 3.

Given a list of  $n$  blocks, each block  $i$  is characterized by the following properties: the allocation time without preloading  $s_i$ , the de-allocation time  $e_i$ , the memory footprint  $m_i$ , and the memory type  $t_i$  (e.g., weights, activations, or intermediates). Specifically, activations denote the binary spike-wise inputs and outputs across network layers. Contrarily, intermediates represent the computational states within individual layers, such as the membrane potential traces of LIF neurons, which are distinct from activations. As discussed in Section §3.2, each layer is partitioned into sub-layers, which are treated as independent layers in the memory planning task, and thus assigned distinct block (layer) IDs.

The objective of the memory planning task is to derive an optimal memory layout, represented as a set of time–address pairs  $\Omega = \{(time_i, addr_i)\}$ , that minimizes inference latency subject to the given constraints, where  $time_i$  denotes the preloading allocation time of block  $i$  and  $addr_i$  denotes its assigned memory address. Meanwhile, the constraints are elaborated as follows:

- (a) *Lifecycle Constraints:*  $1 \leq time_i \leq s_i$  for all weight blocks (allowing preloading), and  $time_i = s_i$  for all other blocks.
- (b) *Memory Address Constraints:* For any block  $i$ , its allocated address must satisfy  $0 \leq addr_i \leq M_{\max} - m_i$ .
- (c) *Non-Overlapping Constraints:* For any pair of blocks  $i$  and  $j$ , their lifecycles or memory regions must not overlap, i.e.,  $(time_i \leq e_j) \vee (time_j \leq e_i)$  or  $(addr_i + m_i \leq addr_j) \vee (addr_j + m_j \leq addr_i)$ .
- (d) *Memory Usage Constraints:* At any point  $v$  in the execution order (x-axis), the total allocated memory must not exceed the device budget, i.e.,  $\sum_{i: time_i \leq v \leq e_i} m_i \leq M_{\max}$ , where  $time_i$  and  $e_i$  denote the allocation and deallocation times of block  $i$ , respectively, and  $M_{\max}$  is the memory budget.

**Memory Planning Algorithm.** As shown in Algorithm 1, we propose a lightweight memory planning workflow for the scheduler that exploits the memory-access patterns of SNN inference to minimize both fragmentation and I/O waiting time. The scheduler operates on a unified plan buffer  $\Omega$  and decomposes the memory planning tasks according to block type, as detailed below:

(i) **PLANMEMBRANEPOTENTIAL:** MASI employs a timestep-agnostic memory planning strategy, first assessing the total memory footprint of all LIF neurons in the SNN and placing their membrane potentials at both ends of the memory buffer. This step is crucial because, unlike ANNs, SNN inference is recurrent across timesteps. However, memory planning cannot be performed on a per-timestep basis: customizing a layout for each timestep is more costly than in conventional ANNs, and the membrane potentials must be preserved throughout all timesteps, which can easily lead to memory fragmentation and incorrect inference due to timestep-aware memory conflicts. As illustrated in Figure 4, under conventional planning, input activations, weights, and the LIF neuronal states of a spike-wise layer with ID 1 would be loaded sequentially. This works correctly at the first timestep, where the SNN behaves similarly to an ANN. Once loaded, the membrane potential records of this layer must be preserved in memory until the final timestep. At the second timestep, these records remain in memory and are not released. However, because the layout plan is timestep-agnostic and reuses the plan derived from the first timestep, conflicts arise between the weights and the preserved membrane potential records of layer 1, ultimately leading to incorrect inference.

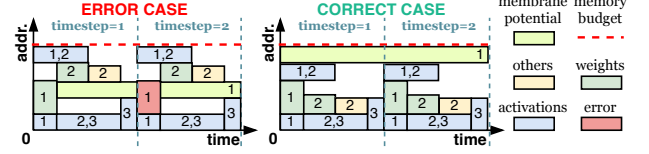
(ii) **PLANACTIVATIONS:** After allocating memory for the LIF neurons' membrane potential records, MASI next prioritizes the planning of activations due to their relatively long lifecycle. Compared to other blocks in the same layer, activations persist longer since they typically serve as both the output of one layer and the input to one or more subsequent layers. By giving activations higher priority and placing them at both boundaries of the memory buffer, MASI effectively reduces fragmentation and preserves a continuous memory region in the middle for other blocks.

(iii) **PLANWITHPRELOAD:** For each layer, MASI allocates weights and other intermediates after activations. It greedily searches for available memory regions that minimize the allocation time of weights, thereby reducing I/O waiting overhead. Once the weight layout is determined, MASI fills the remaining memory with intermediates in a greedy manner.

(iv) **PLANNOPRELOAD:** Since greedy weight preloading can introduce fragmentation and sometimes prevent feasible planning of intermediates, MASI rolls back the layout and reschedules the weights and intermediates of the current layer without preloading. This fallback mechanism ensures a valid memory layout is produced whenever possible.

### 3.4 Timestep-Aware Execution with Early Exit

After determining the memory layout in the offline planning stage, MASI proceeds to the online execution stage, which performs the actual SNN inference in accordance with the plan. Specifically, it decomposes the spike-wise layer execution at each timestep into loading and computing tasks and orchestrates their order through layer dependencies. These tasks are executed by dedicated loading



**Figure 4: A timestep-agnostic planning strategy that allocates membrane potential values first, thereby avoiding errors that may accumulate when SNN inference is executed recurrently across timesteps. The numbers inside the blocks indicate the layer IDs to which they belong.**

and computing threads, each with its own task queue. Execution correctness across layers is ensured by two dependency-based synchronization rules enforced through queue management: (1) the computing task of a layer always waits for its loading task, and (2) when two layers share overlapping memory addresses, the loading task of the latter waits for the computing task of the former.

While the above strategy guarantees correctness within each timestep, the number of timesteps  $T$  inherently induces a correctness-efficiency tradeoff. Increasing  $T$  typically improves accuracy but incurs higher latency and energy cost, whereas reducing  $T$  enhances efficiency at the risk of accuracy degradation [43]. This calls for adaptive timestep configurations for different input samples.

We first consider an SNN model  $f_T$  with a maximum of  $T$  timesteps. If  $f_T$  can correctly predict a sample  $\mathbf{x}$  using  $t$  timesteps, then it will also yield the correct prediction for any  $t'$  satisfying  $t \leq t' \leq T$ . This property highlights the *inclusiveness* of predictions across different timesteps. Let  $\mathcal{A}_t$  be the set of correct predicted input samples for timestep  $t$ , we have  $\mathcal{A}_1 \subseteq \dots \subseteq \mathcal{A}_t \subseteq \dots \subseteq \mathcal{A}_T$ . We adopt the averaged earliest timestep (AET) metric [43] to quantify an ideal scenario in which each correctly classified sample is inferred using the minimum number of timesteps required while preserving overall accuracy. Formally, AET is defined as:

$$\text{AET} = \frac{1}{N} \left( |\mathcal{A}_1| + T|\mathcal{W}| + \sum_{t=2}^T t(|\mathcal{A}_t| - |\mathcal{A}_{t-1}|) \right), \quad (1)$$

where  $\mathcal{W} = \bar{\mathcal{A}}_T$  denotes the set of misclassified samples,  $|\cdot|$  indicates the cardinality of a set, and  $N = |\mathcal{A}_T| + |\mathcal{W}|$  is the total number of samples in the test dataset. For samples in  $\mathcal{W}$ , the maximum number of timesteps  $T$  is assumed, as it is generally not possible to determine a priori whether a sample can be correctly classified.

Hypothetically, each input sample can be associated with a particular *difficulty* level. An *early-exit* SNN can estimate the difficulty of a given input and accordingly determine the number of timesteps required for inference. In this way, the model skips redundant executed timesteps for *easy* samples, ensuring average timesteps converge to AET. Specifically, MASI employs a proxy signal to assess the difficulty of each input sample for the SNN model—namely, the confidence score. Unlike approaches that rely on additional deep policy networks [9, 13, 43], this simple score-based method provides a lightweight and efficient alternative without incurring significant extra memory or computational overhead, making it particularly well-suited for deployment in memory-constrained environments. Specifically, let the model's predicted probability distribution at timestep  $t$  be  $\mathbf{p}^{(t)} = \text{softmax}(\sum_{\tau=1}^t f_{\tau}(\mathbf{x})) = [p_1^{(t)}, p_2^{(t)}, \dots, p_C^{(t)}]$ , where  $C$  denotes the number of object classes. The confidence score

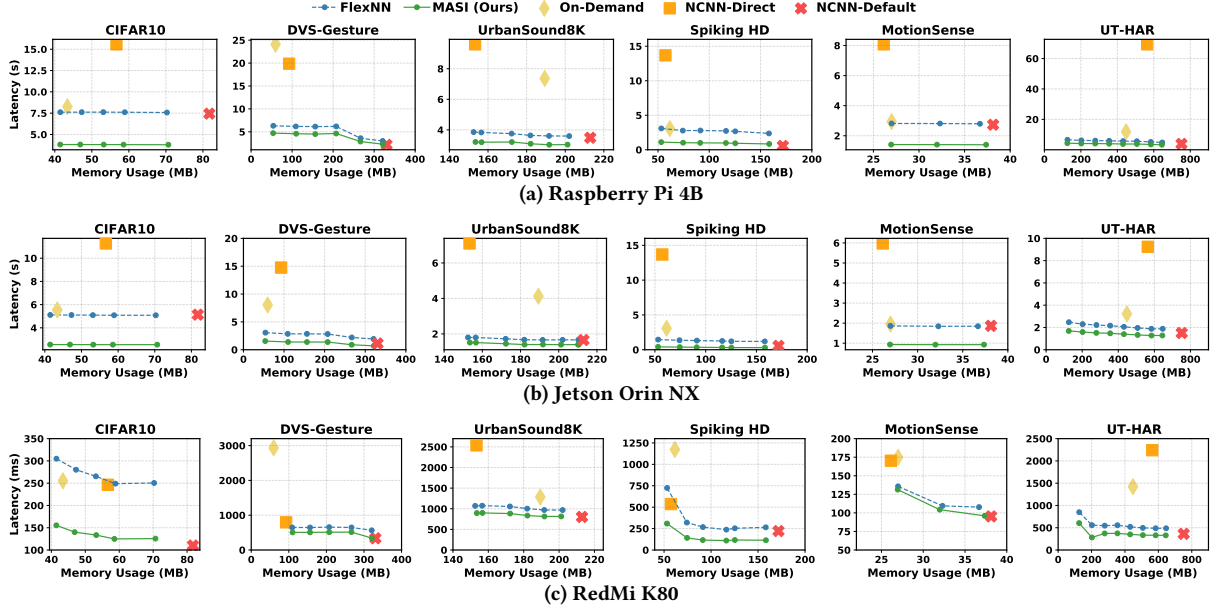


Figure 5: End-to-end latency and memory results across diverse devices and tasks.

Table 1: Summary of experimental settings.

| Sensing  | Dataset           | Model Architecture | #Params | #Timestep |
|----------|-------------------|--------------------|---------|-----------|
| Vision   | CIFAR10 [35]      | Spikformer-4-384   | 9.32M   | 4         |
|          | DVS-Gesture [1]   | SEW-ResNet-34      | 21.29M  | 16        |
| Acoustic | UrbanSound8K [57] | SpikingVGG-9       | 31.61M  | 4         |
|          | Spiking HD [12]   | SEW-ResNet-18      | 11.18M  | 10        |
| Motion   | MotionSense [50]  | Spikformer-2-256   | 2.57M   | 4         |
| Wireless | UT-HAR [78]       | SpikingVGG-16      | 70.32M  | 4         |

till timestep  $t$  is:

$$s_c(\mathbf{x}) = \frac{1}{T} \sum_{\tau=1}^t \mathbf{1}(\hat{y}^{(\tau)} = c), \quad c = 1, 2, \dots, C \quad (2)$$

where  $\hat{y}^{(\tau)} = \arg \max_c p_c^{(\tau)}$  is the index of the maximum probability at timestep  $\tau$ . The score  $s_c \in [0, 1]$  reflects the cumulative frequency with which class  $c$  attains the maximum probability till timestep  $t$ , serving as a proxy for uncertainty. A high confidence score indicates that the prediction distribution is concentrated and thus highly deterministic, whereas a low confidence score suggests a more uniform distribution, implying marked uncertainty. Specifically, we adopt a simple thresholding mechanism: given a preset threshold  $\beta \in [0, 1]$ , the model iteratively evaluates each timestep, and once the confidence score exceeds  $\beta$ , the accumulated output is immediately used for prediction.

Under strict memory constraints, MASI’s slicing and memory scheduling strategies can increase inference latency due to the more frequent operations required for sliced weights and intermediates. Although accuracy is preserved, this overhead may degrade the overall accuracy–efficiency balance. By incorporating the timestep-aware early-exit mechanism, MASI eliminates redundant computation for easy inputs, thereby reducing latency—often below that of the original implementation—and achieving a more favorable trade-off between accuracy and efficiency.

Table 2: Device specification list in our evaluation.

| Device Name     | CPUs                   | RAM (GB) |
|-----------------|------------------------|----------|
| Raspberry Pi 4B | 4×1.8 GHz Cortex-A72   | 8        |
| RedMi K80       | 1×3.3 GHz Cortex-X4    | 12       |
|                 | 5×3.2 GHz Cortex-A720  |          |
|                 | 2×2.8 GHz Cortex-A710  |          |
| Jetson Orin NX  | 2×2.3 GHz Cortex-A520  | 16       |
|                 | 8×2.0 GHz Cortex-A78AE |          |

## 4 Evaluation

### 4.1 Experimental Settings

**Datasets and Models.** The evaluation covers representative mobile intelligence tasks motivated by real-world application scenarios, as mobile/edge devices commonly integrate diverse sensors, including cameras (vision), mmWave/Wi-Fi modules (wireless), microphones/speakers (acoustic), and accelerometers/gyroscopes (motion). To demonstrate the applicability and scalability of MASI, we evaluate it on advanced SNN models, including SpikingVGG (VGG-structured) [79], SEW-ResNet (ResNet-structured) [21], and Spikformer (spike-based Transformer) [86], as summarized in Table 1. The default number of timesteps follows prior work [45, 79, 86].

**Platforms.** MASI is evaluated on a diverse set of edge devices spanning different hardware architectures, all equipped with ARMv8a CPUs, the de facto standard in modern smartphones and widely deployed across edge platforms. The detailed specifications of the experimental edge platforms are summarized in Table 2. The Jetson Orin NX is adopted as the default experimental platform. Specifically, MASI is configured to utilize two CPU cores by default, with computation prioritized on the highest-performance core and the secondary core dedicated to managing weight-loading tasks.

**Metrics.** We evaluate MASI’s real-world performance across multiple metrics: accuracy, memory footprint, end-to-end inference latency, and per-sample energy consumption. Real-time power is monitored via the Jetson device’s built-in sensors [47], with energy

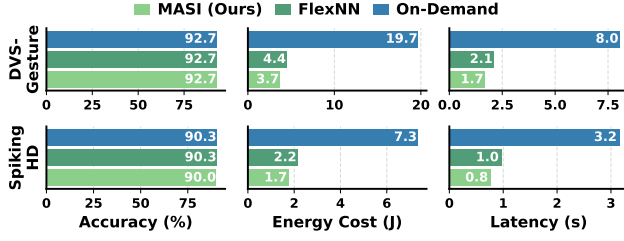


Figure 6: End-to-end performance of baselines.

computed by integrating power over inference time. Accuracy is calculated as the fraction of correctly classified samples per dataset, and memory usage is profiled using the Linux `pmap` command. Latency is measured from sample input to model output.

**Baselines.** All baselines and the MASI prototype are implemented on top of NCNN under a unified evaluation setup for two reasons: (i) NCNN is one of the most efficient mobile inference frameworks, optimized for Arm CPUs and widely adopted in edge intelligence applications; and (ii) to the best of our knowledge, MASI is the first mobile framework explicitly tailored for SNN inference on memory-constrained edge devices, leaving no directly comparable prior work. Accordingly, we construct the following baselines: *NCNN-Default*, which applies NCNN’s standard configuration for ideal on-device SNN inference; *NCNN-Direct*, which disables Winograd and Im2col+GEMM kernels to reduce memory usage; *On-Demand*, which introduces layer-wise swapping and on-demand loading atop NCNN; and *FlexNN* [41], a memory-prioritized mobile inference framework originally for ANNs adapted to SNNs.

## 4.2 End-to-End Performance

The end-to-end evaluation spans three edge devices and six SNN models. In Figure 5, MASI and FlexNN are shown as curves, where each point denotes a memory–latency pair under a given constraint, while other baselines appear as fixed points due to static memory usage. The red X marks NCNN-default, which is infeasible on edge devices as it indicates the unoptimized maximum memory footprint.

For peak memory usage, the left end of each curve indicates the minimum budget supported by FlexNN and MASI. Across all cases, MASI requires less memory than NCNN-Default, and in most cases also outperforms On-Demand and NCNN-Direct. On Raspberry Pi 4B, MASI reduces memory usage by up to 86.65% on SEW-ResNet, 57.29% on SpikingVGG, and 49.28% on Spikformer models, respectively. While On-Demand lowers memory through basic layer streaming, MASI’s memory-adaptive slicing further decreases layer-wise footprint, achieving the lowest overall memory usage.

Beyond memory savings, MASI also lowers inference latency, improving efficiency across edge devices. Under constrained budgets, it sustains lower or comparable latency to NCNN-Default. Moreover, under sufficient budgets, it can even outperform it by retaining larger weight slices in memory and avoiding I/O stalls via effective memory planning strategies. Early exit further reduces redundant computation, yielding significant gains—for instance, latency drops by up to 14.83% on SpikingVGG, 25.49% on SEW-ResNet, and 49.15% on Spikformer, separately, on the Jetson Orin NX. On Raspberry Pi, MASI delivers slightly weaker improvements than on other devices due to a shifted system bottleneck. While SNN inference is generally compute-bound—enabling MASI to overlap

Table 3: Fine-grained storage and time overheads of MASI.

| Overhead             | DVS-Gesture | UrbanSound8K | Spiking HD | UT-HAR  |
|----------------------|-------------|--------------|------------|---------|
| Memory Budget (MB)   | 195         | 150          | 90         | 300     |
| Weights Storage (MB) | 319.74      | 142.84       | 157.69     | 683.78  |
| Slicing Cost (ms)    | 1401.79     | 2417.98      | 785.80     | 2798.67 |
| Profiling Cost (ms)  | 153.40      | 480.00       | 124.34     | 715.33  |
| Planning Cost (ms)   | 352.66      | 5.78         | 68.51      | 503.29  |

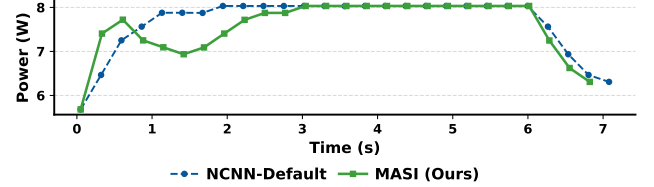


Figure 7: Real-time power consumption of MASI and NCNN-Default when processing eight DVS-Gesture inputs.

weight loading with computation—the Pi’s limited I/O bandwidth (257 MB/s vs. 2481 MB/s on Redmi K80) makes inference I/O-bound, thereby limiting further latency reduction. Figure 6 further presents detailed comparisons under memory constraints for our selected neuromorphic sensing tasks, where NCNN-based implementations fail. Results show that MASI sustains accuracy comparable to other baselines when  $\beta$  is set to 0.6. With memory management and slice preloading, both MASI and FlexNN reduce 3–6 $\times$  energy costs and inference latency. Beyond this, MASI achieves additional improvements, saving 15.9% energy and 19.1% latency on DVS-Gesture, and 22.7% energy and 20.0% latency on Spiking HD.

## 4.3 System Overhead

The overhead of MASI arises from two primary sources: offline planning for optimal layout and runtime I/O for weight preloading. **Offline Planning Overhead.** The offline overhead includes storing pre-transformed slices, model profiling, layer weights slicing, and memory layout planning. Storage is not a bottleneck since sliced weights are MB-level, while device storage can be expanded to tens or hundreds of GB via SD cards. Time overhead is negligible: profiling and slicing occur once at initialization, and layout planning is invoked only when budgets change. As shown in Table 3, adaptation consistently completes within one second, which is fast enough for responsiveness-critical scenarios. Re-slicing and re-planning are necessary only if a new budget invalidates prior slicing or an allocation error occurs, both of which are rare under our constrained memory design.

**Runtime I/O Overhead.** The runtime overhead of MASI stems primarily from additional I/O for sliced weights. As shown in Figure 7, it sustains peak power comparable to NCNN while reducing latency, resulting in a 2.82% energy saving in the DVS-Gesture case. Leveraging early-exit to trim redundant computation, MASI bounds the I/O overhead under tight memory budgets, ensuring its total energy cost does not exceed that of the ideal NCNN baseline.

## 4.4 Real-Time Memory Adaptation

Given that devices may concurrently run applications or services during SNN inference, the available memory for inference can vary over time and even abruptly change, leading to inference

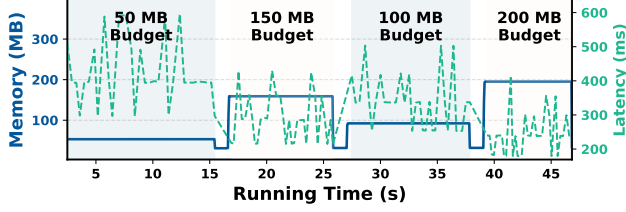


Figure 8: Real-time latency and memory usage of MASI under changing memory budgets.

interruptions caused by insufficient memory. Figure 8 reports the real-time memory usage and inference latency of MASI under four sequentially changing memory budgets. Upon a budget change, MASI pauses inference, releases unnecessary memory, generates a new execution plan, reallocates memory, and resumes, with the entire process completing in about 1 second. MASI precisely satisfies the target budgets and achieves lower average latency under larger budgets. In practice, memory reallocation would be triggered by developer-defined policies, which lie beyond the scope of this paper.

#### 4.5 Ablation Study

**Impact of Components on Inference Acceleration.** Figure 9 presents the ablation results, where we evaluate the contributions of layout planning, parallel preloading, and timestep-aware early exit to latency reduction. Under constrained budgets, integrating optimized memory layout with early exit achieves over 2× speedup, and parallel preloading offers smaller but steady gains, highlighting that MASI’s components offer orthogonal acceleration gains.

**Comparison of Confidence Score.** We benchmark the confidence score of MASI against three conventional threshold measures: (1) *MaxProb* [26], the maximum predicted probability, (2) *Margin* [4], the gap between the top-2 probabilities, and (3) normalized *Entropy* [66], computed from the output distribution. As shown in Figure 10, the cumulative confidence score in MASI more reliably identifies the required timesteps for different inputs to preserve accuracy, outperforming other methods by at least 17.99% on average.

### 5 RELATED WORK

**Lightweight Support for SNN.** Prior work on enabling deep SNN inference for resource-constrained devices has concentrated on algorithm-level efficiency, including pruning [10, 44, 60, 88], quantization [56, 72], manually crafted efficient architectures [11, 42], and NAS [33, 34, 46, 52]. Pruning reduces memory and computation by eliminating redundant parameters and structures, while quantization compresses models via low-precision representations. NAS, in contrast, automatically identifies architectures optimized for task and platform constraints. Despite their effectiveness, these approaches often compromise performance and lack robustness under dynamic memory budgets, underscoring the need for lightweight system-level mechanisms to ensure reliable SNN inference on edge devices. While prior ANN-based studies offer useful references, such as partitioning and swapping layers between memory and storage [5, 37, 59], or memory scheduling under constraints [41, 68, 77]. Directly applying them to SNNs is suboptimal due to the distinct computing paradigm. This gap motivates SNN-aware

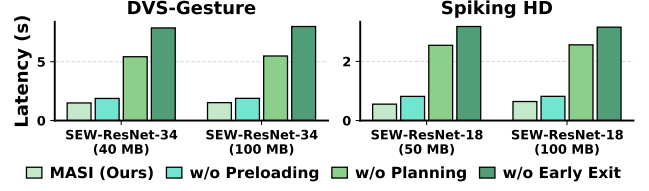


Figure 9: Ablation of MASI’s inference acceleration modules under varying memory budgets.

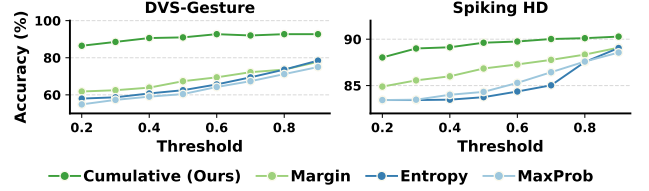


Figure 10: Comparison of confidence score policy.

system-level solutions that complement algorithmic compression to ensure efficient and dependable inference.

**Conditional Computing for SNN.** Conditional computing approaches have been shown to enhance model expressiveness by dynamically adapting activations, parameters, or architectures according to individual input samples [25]. This paradigm is particularly relevant for SNNs, whose time-dependent activations vary with each input, naturally motivating research on accuracy–efficiency trade-offs. Specifically, SEENN [43] determines the exit time step based on confidence scores or a policy network. Meanwhile, another confidence-based dynamic SNN [39] identifies the optimal confidence threshold via a Pareto front analysis. MTT [18] further enhances temporal flexibility in SNNs to enable efficient deployment with low latency in fully event-driven simulations. Given that memory-constrained deployment inherently involves trade-offs with inference latency [41, 79], per-sample adaptive computation provides a mechanism to mitigate the latency overheads of memory-efficient strategies. Motivated by this, we propose input-dependent, timestep-aware computation in SNNs to enable efficient edge deployment while minimizing latency.

### 6 Conclusion

In this paper, we presented MASI, an adaptive memory management framework for efficient on-device SNN inference under tight memory constraints. By jointly planning slicing, loading, and computation, MASI achieves near-optimal memory utilization with minimal management overhead. Evaluations show that MASI effectively adapts to varying memory budgets while maintaining favorable accuracy–efficiency trade-offs, providing a practical foundation for energy-efficient SNN deployment on heterogeneous edge devices. The limitations and future directions are discussed in **Appendix B**.

#### Acknowledgments

The work of this paper is supported by the National Key Research and Development Program of China under Grant No.2022YFB4500100, the National Natural Science Foundation of China under Grant No.62502443, No.62125206, and the Zhejiang Provincial Natural Science Foundation of China under Grant No. LD24F020014.

## References

- [1] Arnon Amir, Brian Taba, David Berg, Timothy Melano, Jeffrey McKinstry, Carmelo Di Nolfo, Tapan Nayak, Alexander Andreopoulos, Guillaume Garreau, Marcela Mendoza, et al. 2017. A low power, fully event-based gesture recognition system. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 7243–7252.
- [2] Abdulla Ayyad, Mohamad Halwani, Dewald Swart, Rajkumar Muthusamy, Fahad Almaskari, and Yahya Zweiri. 2023. Neuromorphic vision based control for the precise positioning of robotic drilling systems. *Robotics and Computer-Integrated Manufacturing* 79 (2023), 102419.
- [3] Velibor Bojkovic, Xiaofeng Wu, and Bin Gu. 2025. Temporal Misalignment in ANN-SNN Conversion and its Mitigation via Probabilistic Spiking Neurons. In *Forty-second International Conference on Machine Learning*.
- [4] Tolga Bolukbasi, Joseph Wang, Ofer Dekel, and Venkatesh Saligrama. 2017. Adaptive neural networks for efficient inference. In *International conference on machine learning*. PMLR, 527–536.
- [5] Qixuan Cai, Ruikai Chu, Kaixuan Zhang, Xiulong Liu, Xinyu Tong, Xin Xie, Jiancheng Chen, and Keqiu Li. 2025. AMRE: Adaptive Multilevel Redundancy Elimination for Multimodal Mobile Inference. *IEEE Transactions on Mobile Computing* (2025).
- [6] Yun-Hin Chan, Rui Zhou, Running Zhao, Zhihan JIANG, and Edith C. H. Ngai. 2024. Internal Cross-layer Gradients for Extending Homogeneity to Heterogeneity in Federated Learning. In *The Twelfth International Conference on Learning Representations*.
- [7] Zhaoxin Chang, Fusang Zhang, Jie Xiong, Weiyan Chen, and Daqing Zhang. 2024. MSense: Boosting wireless sensing capability under motion interference. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 108–123.
- [8] Kumar Chellapilla, Sidd Puri, and Patrice Simard. 2006. High performance convolutional neural networks for document processing. In *Tenth international workshop on frontiers in handwriting recognition*. Suvisoft.
- [9] Yichong Chen, Zifeng Niu, Manuel Roveri, and Giuliano Casale. 2025. CEED: Collaborative Early Exit Neural Network Inference at the Edge. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [10] Yanqi Chen, Zhaofei Yu, Wei Fang, Zhengyu Ma, Tiejun Huang, and Yonghong Tian. 2022. State transition of dendritic spines improves learning of sparse spiking neural networks. In *International Conference on Machine Learning*. PMLR, 3701–3715.
- [11] Loic Cordone, Benoît Miramond, and Philippe Thierion. 2022. Object detection with spiking neural networks on automotive event data. In *2022 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.
- [12] Benjamin Cramer, Yannik Stradmann, Johannes Schemmel, and Friedemann Zenke. 2020. The heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems* 33, 7 (2020), 2744–2757.
- [13] Xin Dai, Xiangnan Kong, and Tian Guo. 2020. EPNet: Learning to exit with flexible multi-branch network. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 235–244.
- [14] Lei Deng, Yujie Wu, Xing Hu, Ling Liang, Yufei Ding, Guoqi Li, Guangshe Zhao, Peng Li, and Yuan Xie. 2020. Rethinking the performance comparison between SNNs and ANNs. *Neural networks* 121 (2020), 294–307.
- [15] Shuiguang Deng, Hailiang Zhao, Weijia Fang, Jianwei Yin, Schahram Dustdar, and Albert Y Zomaya. 2020. Edge intelligence: The confluence of edge computing and artificial intelligence. *IEEE Internet of Things Journal* 7, 8 (2020), 7457–7469.
- [16] Android Developers. 2023. Manage your app's memory. <https://developer.android.com/topic/performance/memory>. Accessed: 2023-08-01.
- [17] Manuel F Dolz, Héctor Martínez, Adrián Castelló, Pedro Alonso-Jordá, and Enrique S Quintana-Orti. 2023. Efficient and portable Winograd convolutions for multi-core processors. *The Journal of Supercomputing* 79, 10 (2023), 10589–10610.
- [18] Kangrui Du, Yuhang Wu, Shikuang Deng, and Shi Gu. 2025. Temporal Flexibility in Spiking Neural Networks: Towards Generalization Across Time Steps and Deployment Friendliness. In *The Thirteenth International Conference on Learning Representations*.
- [19] Maryam Ebrahimi, Alexandre da Silva Veith, Moshe Gabel, and Eyal de Lara. 2022. Combining DNN partitioning and early exit. In *Proceedings of the 5th International Workshop on Edge Systems, Analytics and Networking*. 25–30.
- [20] Yimeng Fan, Wei Zhang, Changsong Liu, Mingyang Li, and Wenrui Lu. 2024. Sfod: Spiking fusion object detector. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 17191–17200.
- [21] Wei Fang, Zhaofei Yu, Yanqi Chen, Tiejun Huang, Timothée Masquelier, and Yonghong Tian. 2021. Deep residual learning in spiking neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 21056–21069.
- [22] Charlotte Frenkel, Martin Lefebvre, Jean-Didier Legat, and David Bol. 2018. A 0.086-mm<sup>2</sup> 12.7-pJ/SOP 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm CMOS. *IEEE transactions on biomedical circuits and systems* 13, 1 (2018), 145–158.
- [23] Wulfram Gerstner, Werner M Kistler, Richard Naud, and Liam Paninski. 2014. *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press.
- [24] Samanwoy Ghosh-Dastidar and Hojjat Adeli. 2009. Spiking neural networks. *International journal of neural systems* 19, 04 (2009), 295–308.
- [25] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. 2021. Dynamic neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence* 44, 11 (2021), 7436–7456.
- [26] Dan Hendrycks and Kevin Gimpel. 2017. A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks. In *International Conference on Learning Representations*.
- [27] Alan L Hodgkin and Andrew F Huxley. 1952. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of physiology* 117, 4 (1952), 500.
- [28] Hongsheng Hu, Zoran Salcic, Lichao Sun, Gillian Dobbie, Philip S Yu, and Xuyun Zhang. 2022. Membership inference attacks on machine learning: A survey. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–37.
- [29] Xiaoqian Huang, Rajkumar Muthusamy, Eman Hassan, Zhenwei Niu, Lakmal Seneviratne, Dongming Gan, and Yahya Zweiri. 2020. Neuromorphic vision based contact-level classification in robotic grasping applications. *Sensors* 20, 17 (2020), 4724.
- [30] Zihan Huang, Wei Fang, Tong Bu, Peng Xue, Zecheng Hao, Wenxuan Liu, Yuanhong Tang, Zhaofei Yu, and Tiejun Huang. 2025. Differential Coding for Training-Free ANN-to-SNN Conversion. In *Forty-second International Conference on Machine Learning*.
- [31] Andrey Ignatov, Radu Timofte, William Chou, Ke Wang, Max Wu, Tim Hartley, and Luc Van Gool. 2018. Ai benchmark: Running deep neural networks on android smartphones. In *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*. 0–0.
- [32] Eugene M Izhikevich. 2003. Simple model of spiking neurons. *IEEE Transactions on neural networks* 14, 6 (2003), 1569–1572.
- [33] Youngeun Kim, Yuhang Li, Hyoungseob Park, Yeshwanth Venkatesha, and Priyadarshini Panda. 2022. Neural architecture search for spiking neural networks. In *European conference on computer vision*. Springer, 36–56.
- [34] Youngeun Kim, Yuhang Li, Hyoungseob Park, Yeshwanth Venkatesha, Ruokai Yin, and Priyadarshini Panda. 2022. Exploring lottery ticket hypothesis in spiking neural networks. In *European Conference on Computer Vision*. Springer, 102–120.
- [35] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).
- [36] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).
- [37] Taegyeong Lee, Zhiqi Lin, Saumay Pushp, Caihua Li, Yunxin Liu, Youngki Lee, Fengyuan Xu, Chenren Xu, Lintao Zhang, and Junehwa Song. 2019. Occlumency: Privacy-preserving remote deep-learning inference using SGX. In *The 25th Annual international conference on mobile computing and networking*. 1–17.
- [38] Claudia Lenk, Philipp Hövel, Kalpan Ved, Steve Durstewitz, Thomas Meurer, Tobias Fritsch, Andreas Männchen, Jan Küller, Daniel Beer, Tzvetan Ivanov, et al. 2023. Neuromorphic acoustic sensing using an adaptive microelectromechanical cochlea with integrated feedback. *Nature electronics* 6, 5 (2023), 370–380.
- [39] Chen Li, Edward G Jones, and Steve Furber. 2023. Unleashing the potential of spiking neural networks with dynamic confidence. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 13350–13360.
- [40] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li, and Luping Shi. 2017. Cifar10-dvs: an event-stream dataset for object classification. *Frontiers in neuroscience* 11 (2017), 244131.
- [41] Xiangyu Li, Yuanchun Li, Yuanzhe Li, Ting Cao, and Yunxin Liu. 2024. Flexnn: Efficient and adaptive dnn inference on memory-constrained edge devices. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 709–723.
- [42] Yuhang Li, Shikuang Deng, Xin Dong, Ruihao Gong, and Shi Gu. 2021. A free lunch from ANN: Towards efficient, accurate spiking neural networks calibration. In *International conference on machine learning*. PMLR, 6316–6325.
- [43] Yuhang Li, Tamar Geller, Youngeun Kim, and Priyadarshini Panda. 2023. Seenn: Towards temporal spiking early exit neural networks. *Advances in Neural Information Processing Systems* 36 (2023), 63327–63342.
- [44] Yaxin Li, Qi Xu, Jiangrong Shen, Hongming Xu, Long Chen, and Gang Pan. 2024. Towards efficient deep spiking neural networks construction with spiking activity based pruning. In *International Conference on Machine Learning*. PMLR, 29063–29073.
- [45] Yuhang Li, Ruokai Yin, Youngeun Kim, and Priyadarshini Panda. 2023. Efficient human activity recognition with spatio-temporal spiking neural networks. *Frontiers in Neuroscience* 17 (2023), 1233037.
- [46] Qianhui Liu, Jiaqi Yan, Malu Zhang, Gang Pan, and Haizhou Li. 2024. LitE-SNN: designing lightweight and efficient spiking neural network through spatial-temporal compressive network search and joint optimization. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 3097–3105.

- [47] Xinyue Ma, Suyeon Jeong, Minjia Zhang, Di Wang, Jonghyun Choi, and Myeongjae Jeon. 2023. Cost-effective on-device continual learning over memory hierarchy with Miro. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*. 1–15.
- [48] Martin Maas, Ulysse Beaugnon, Arun Chauhan, and Berkin Ilbeyli. 2022. Talamalloc: Efficient on-chip memory allocation for production machine learning accelerators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 123–137.
- [49] Wolfgang Maass. 1997. Networks of spiking neurons: the third generation of neural network models. *Neural networks* 10, 9 (1997), 1659–1671.
- [50] Mohammad Malekzadeh, Richard G Clegg, Andrea Cavallaro, and Hamed Hadadi. 2019. Mobile sensor data anonymization. In *Proceedings of the international conference on internet of things design and implementation*. 49–58.
- [51] Dylan Richard Muir and Sadique Sheik. 2025. The road to commercial success for neuromorphic technologies. *Nature communications* 16, 1 (2025), 3586.
- [52] Byunggook Na, Jisoo Mok, Seongsik Park, Dongjin Lee, Hyeokjun Choe, and Sungroh Yoon. 2022. Autosnn: Towards energy-efficient spiking neural networks. In *International conference on machine learning*. PMLR, 16253–16269.
- [53] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. 2019. Knockoff nets: Stealing functionality of black-box models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 4954–4963.
- [54] Francesco Paissan, Massimo Gottardi, and Elisabetta Farella. [n. d.]. Enabling energy efficient machine learning on a Ultra-Low-Power vision sensor for IoT. *System-level Design Methods for Deep Learning on Heterogeneous Architectures (SLOHA 2021)* ([n. d.]), 1.
- [55] Bo Ren Pao, I-Chia Chen, En-Hao Chang, and Tsung Tai Yeh. 2025. EDA: Energy-Efficient Inter-Layer Model Compilation for Edge DNN Inference Acceleration. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 563–576.
- [56] Xuerui Qiu, Malu Zhang, Jieyuan Zhang, Wenjie Wei, Honglin Cao, Junsheng Guo, Rui-Jie Zhu, Yimeng Shan, Yang Yang, and Haizhou Li. 2025. Quantized Spike-driven Transformer. In *The Thirteenth International Conference on Learning Representations*.
- [57] Justin Salamon, Christopher Jacoby, and Juan Pablo Bello. 2014. A dataset and taxonomy for urban sound research. In *Proceedings of the 22nd ACM international conference on Multimedia*. 1041–1044.
- [58] Samsung. 2025. Samsung Galaxy S Series - Official Search. <https://www.samsung.com/us/smartphones/>. Accessed: 2025-08-29.
- [59] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [60] Xinyu Shi, Jianhao Ding, Zecheng Hao, and Zhaofei Yu. 2024. Towards energy efficient spiking neural networks: An unstructured pruning framework. In *The Twelfth International Conference on Learning Representations*.
- [61] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606.
- [62] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S Emer. 2017. Efficient processing of deep neural networks: A tutorial and survey. *Proc. IEEE* 105, 12 (2017), 2295–2329.
- [63] Kaiwen Tang, Zhanglu Yan, and Weng-Fai Wong. 2025. Sorbet: A Neuromorphic Hardware-Compatible Transformer-Based Spiking Language Model. In *Forty-second International Conference on Machine Learning*.
- [64] Ge Tao, Ting Cao, Si-Qing Chen, and Qiong(Emma) Ning. 2023. Achieving Zero-COGS with Microsoft Editor Neural Grammar Checker. <https://www.microsoft.com/en-us/research/blog/achieving-zero-cogs-with-microsoft-editor-neural-grammar-checker/>. Accessed: 2023-05-18.
- [65] Amirhossein Tavaneai, Masoud Ghodrati, Saeed Reza Kheradpisheh, Timothée Masquelier, and Anthony Maida. 2019. Deep learning in spiking neural networks. *Neural networks* 111 (2019), 47–63.
- [66] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. 2016. Branchynet: Fast inference via early exiting from deep neural networks. In *2016 23rd international conference on pattern recognition (ICPR)*. IEEE, 2464–2469.
- [67] Bernhard Vogginger, Amirhossein Rostami, Vaibhav Jain, Sirine Arfa, Andreas Hantsch, David Kappel, Michael Schäfer, Ulrike Faltings, Hector A Gonzalez, Chen Liu, et al. 2024. Neuromorphic hardware for sustainable AI data centers. *arXiv preprint arXiv:2402.02521* (2024).
- [68] Qipeng Wang, Mengwei Xu, Chao Jin, Xinran Dong, Jinliang Yuan, Xin Jin, Gang Huang, Yunxin Liu, and Xuanzhe Liu. 2022. Melon: Breaking the memory wall for resource-efficient on-device machine learning. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*. 450–463.
- [69] Qingyu Wang, Tieli Zhang, Minglun Han, Yi Wang, Duzhen Zhang, and Bo Xu. 2023. Complex dynamic neurons improved spiking transformer network for efficient automatic speech recognition. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 102–109.
- [70] Shuai Wang, Malu Zhang, Dehao Zhang, Ammar Belatreche, Yichen Xiao, Yu Liang, Yimeng Shan, Qian Sun, Enqi Zhang, and Yang Yang. 2025. Spiking Vision Transformer with Saccadic Attention. In *The Thirteenth International Conference on Learning Representations*.
- [71] Chiyue Wei, Bowen Duan, Cong Guo, Jingyang Zhang, Qingyue Song, Hai Li, and Yiran Chen. 2025. Phi: Leveraging Pattern-based Hierarchical Sparsity for High-Efficiency Spiking Neural Networks. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 930–943.
- [72] Wenjie Wei, Malu Zhang, Zijian Zhou, Ammar Belatreche, Yimeng Shan, Yu Liang, Honglin Cao, Jieyuan Zhang, and Yang Yang. 2025. QP-SNN: Quantized and Pruned Spiking Neural Networks. In *The Thirteenth International Conference on Learning Representations*.
- [73] Xingrun Xing, Boyan Gao, Zheng Liu, David A. Clifton, Shitao Xiao, Wanpeng Zhang, Li Du, Zheng Zhang, Guoqi Li, and Jiajun Zhang. 2025. SpikeLLM: Scaling up Spiking Neural Network to Large Language Models via Saliency-based Spiking. In *The Thirteenth International Conference on Learning Representations*.
- [74] Xingrun Xing, Zheng Zhang, Ziyi Ni, Shitao Xiao, Yiming Ju, Siqi Fan, Yequan Wang, Jiajun Zhang, and Guoqi Li. 2024. SpikeLM: Towards General Spike-Driven Language Modeling via Elastic Bi-Spiking Mechanisms. In *International Conference on Machine Learning*. PMLR, 54698–54714.
- [75] Qi Xu, Jie Deng, Jiangrong Shen, Biwu Chen, Huajin Tang, and Gang Pan. 2025. Hybrid Spiking Vision Transformer for Object Detection with Event Cameras. In *Forty-second International Conference on Machine Learning*.
- [76] Man Yao, Ole Richter, Guangshe Zhao, Ning Qiao, Yinnan Xing, Dingheng Wang, Tianxiang Hu, Wei Fang, Tugba Demirci, Michele De Marchi, et al. 2024. Spike-based dynamic computing with asynchronous sensing-computing neuromorphic chip. *Nature Communications* 15, 1 (2024), 4464.
- [77] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2025. EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices. *IEEE Transactions on Mobile Computing* 24, 8 (2025), 7059–7073.
- [78] Siamak Yousefi, Hirokazu Narui, Sankalp Dayal, Stefano Ermon, and Shahrokh Valaei. 2017. A survey on behavior recognition using WiFi channel state information. *IEEE Communications Magazine* 55, 10 (2017), 98–104.
- [79] Di Yu, Xin Du, Linshan Jiang, Wentao Tong, and Shuiguang Deng. 2024. EC-SNN: Splitting Deep Spiking Neural Networks for Edge Devices. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 5389–5397.
- [80] Dingyi Zeng, Yuchen Wang, Honglin Cao, Wanlong Liu, Yichen Xiao, Wenyu Chen, Malu Zhang, Guoqing Wang, Yang Yang, et al. 2025. Leveraging Asynchronous Spiking Neural Networks for Ultra Efficient Event-Based Visual Processing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 1620–1628.
- [81] Xinran Zhang, Hanqi Zhu, Yifan Duan, Wuyang Zhang, Longfei Shangguan, Yu Zhang, Jianmin Ji, and Yanyong Zhang. 2024. Map++: Towards user-participatory visual SLAM systems with efficient map expansion and sharing. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 633–647.
- [82] Ziqi Zhang, Chen Gong, Yifeng Cai, Yuanyuan Yuan, Bingyan Liu, Ding Li, Yao Guo, and Xiangqun Chen. 2024. No privacy left outside: On the (in-) security of tee-shielded dnn partition for on-device ml. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3327–3345.
- [83] Hong-Sheng Zheng, Yu-Yuan Liu, Chen-Fong Hsu, and Tsung Tai Yeh. 2023. Streamlet: memory-efficient streaming tiny deep learning inference on the microcontroller. *Advances in Neural Information Processing Systems* 36 (2023), 37160–37172.
- [84] Kai Zheng, Kun Qian, Timothy Woodford, and Xinyu Zhang. 2023. Neuroradar: A neuromorphic radar sensor for low-power iot systems. In *Proceedings of the 21st ACM Conference on Embedded Networked Sensor Systems*. 223–236.
- [85] Haozhe Zhou, Sudershan Boovaraghavan, Mayank Goel, and Yuvraj Agarwal. 2024. On-Device Speech Filtering for Privacy-Preserving Acoustic Activity Recognition. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1802–1804.
- [86] Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng YAN, Yonghong Tian, and Li Yuan. 2023. Spikformer: When Spiking Neural Network Meets Transformer. In *The Eleventh International Conference on Learning Representations*.
- [87] Dekai Zhu, Zhongcong Xu, Jinhui Dong, Canbo Ye, Yinbai Hu, Hang Su, Zhengfa Liu, and Guang Chen. 2019. Neuromorphic visual odometry system for intelligent vehicle application with bio-inspired vision sensor. In *2019 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2225–2232.
- [88] Zhengyang Zhuge, Peisong Wang, Xingtang Yao, and Jian Cheng. 2024. Towards Efficient Spiking Transformer: a Token Sparsification Framework for Training and Inference Acceleration. In *Forty-first International Conference on Machine Learning*.

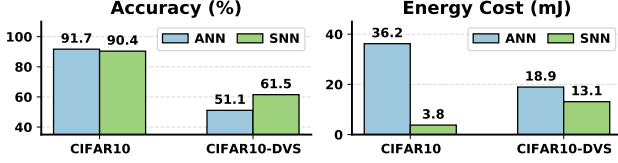


Figure 11: Case study of VGG-9 ANN vs. SNN on CIFAR10 and DVS-Captured CIFAR10 (CIFAR10-DVS). SNNs achieve comparable performance to ANNs on conventional CIFAR10 and outperform them on CIFAR10-DVS, while requiring much lower theoretical computational cost, highlighting their suitability for resource-constrained edge deployment.

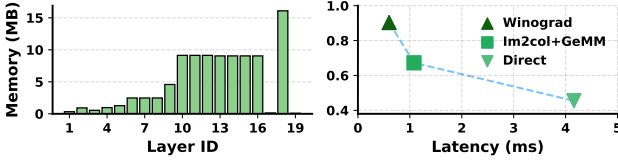


Figure 12: (Left) Layer-wise memory usage of a spike-based VGG-19 model, showing that few layers dominate the peak memory footprint. (Right) Latency-memory trade-off for three 3x3 convolution kernels with 16 channels, illustrating a negative correlation between latency and memory cost.

## A Background Supplementary

### A.1 Case Study for SNN Superiority

To further validate this potential, we perform a controlled comparison between ANNs and SNNs with identical architectures on both standard CIFAR10 and its event-driven variant captured by a DVS. As shown in Figure 11, on CIFAR10, an SNN with a timestep of 4 achieves accuracy comparable to an ANN, with only a 1.3% drop, demonstrating similar feature extraction capability on conventional sensing tasks. In contrast, its theoretical inference energy is roughly 9× lower, owing to the sparse computing paradigm that reduces operations across layers. On CIFAR10-DVS, with a timestep of 10, SNNs outperform ANNs by approximately 10% in accuracy when processing sparse, asynchronous event streams. Inference energy remains lower (5.8 mJ), though the energy gap is smaller than for conventional CIFAR10. This reduction stems from two factors: (i) sparse DVS inputs inherently avoid redundant computation, and (ii) extracting spatiotemporal features from raw events requires a larger timestep, increasing the SNN’s computational cost. Overall, the combination of competitive accuracy and lower inference energy suggests that SNNs are highly suitable for scalable deployment on resource-constrained edge devices.

### A.2 Case Study for Memory Bottleneck

We further perform detailed memory profiling of a representative deep SNN model to implement the CIFAR10 classification task, SpikingVGG-19, as illustrated in Figure 12. Deep SNN models exhibit a highly imbalanced layer-wise memory distribution, with only a few layers reaching peak usage of ~16 MB, while over 50% consume less than 5 MB. Moreover, various implementations exist for each SNN layer to balance latency and memory usage. For convolution layers, common approaches include Direct [36], Im2col+GeMM [8], and Winograd [17]. Unlike Direct convolution,

Winograd transforms weights into memory during the preparation stage, whereas Im2col+GeMM flattens input features into in-memory intermediates. Among them, Winograd achieves the lowest latency but the highest memory consumption. While these intermediate inputs from Im2col+GeMM and pre-transformed weights from Winograd enable low-latency inference, they also become the main memory bottlenecks on resource-constrained devices, providing a natural starting point for memory-aware SNN inference optimization.

## B Discussion and Future Work

We discuss remaining limitations and opportunities for future work on on-device SNN inference.

**Other Acceleration Design.** In §3.4, we presented an early-exit mechanism tailored to the timestep dimension of SNNs, which effectively reduces redundant computation and accelerates inference. This design leverages the intrinsic temporal dynamics of SNNs, making it a specialized form of conditional computing. Beyond our approach, other early-exit techniques—such as branching with redundant model components [19, 66]—can also be integrated into SNN inference. These methods, however, are largely agnostic to neural architectures and apply equally well to both ANNs and SNNs. Since they can be directly transferred without SNN-specific adaptation, we focus our discussion on the timestep-aware mechanism in MASI, and leave the incorporation of more general early-exit strategies to complementary efforts in the future.

**Cross-Platform Compatibility.** While neuromorphic chips [22, 76] could unlock the full energy benefits of SNNs, their limited availability and high cost restrict widespread adoption [51, 67]. MASI demonstrates that efficient SNN inference is achievable on mobile CPUs, validating its practicality for current edge devices. Its generic slicing and planning design allows migration to other backends, such as GPUs, NPUs, TPUs, and neuromorphic chips. Still, deployment must account for potential overheads from data movement, memory management, and scheduler energy consumption. These considerations highlight key directions for future work: optimizing for heterogeneous hardware, reducing I/O and memory costs, and enhancing energy proportionality. Collectively, MASI provides a practical foundation for bringing always-on, energy-efficient SNN intelligence to real-world IoT and mobile applications.

**Neuromorphic Sensing.** Neuromorphic sensors offer a compelling alternative to traditional sensors in IoT environments due to their high energy efficiency, low latency, and fine-grained temporal resolution [2, 29, 87]. Their high cost, however, currently limits large-scale deployment. As advances in design and manufacturing reduce these costs, broader adoption is expected, motivating the integration of SNNs into edge devices. This trend highlights a future direction for edge intelligence: combining cost-effective neuromorphic sensing with energy-efficient SNN inference to enable always-on, low-latency, and scalable IoT applications.

**Large Model Portability.** Recent efforts have begun exploring SNN-based large language models (LLM) [63, 73, 74], which hold potential for enabling energy-efficient large-model services on edge devices. Given the rapid proliferation of LLM applications, an important question is whether MASI can be extended to this domain. While spiking LLMs are still in their infancy, our evaluation with

Spikformer already demonstrates that MASI is compatible with transformer-like structures, suggesting feasibility for future adaptation. A full investigation of this direction is left as future work. Still, we believe MASI offers a practical foundation that can be readily leveraged as spiking LLMs continue to evolve.

**Adversary Considerations.** Although our work centers on memory-efficient deployment of SNNs, on-device models inevitably face security risks in white-box settings, such as model stealing and membership inference attacks [28, 53]. While these concerns are beyond

the scope of this study, our framework is inherently compatible with protection mechanisms like Trusted Execution Environments (TEEs) [37, 82], since our optimization focuses on memory and computation efficiency rather than altering security assumptions. This orthogonality enables the integration of efficiency and security in future extensions.